

CSE 210

[Home](#)[W1](#)[W2](#)[W3](#)[W4](#)[W5](#)[W6](#)[W7](#)

W03 Project: Scripture Memorizer Program

Overview

People often try to memorize poems or passages of scripture. One of the challenges they encounter is that they want to hide the scripture while they are practicing, but they may not be able to recite the whole scripture from memory just yet.

To help solve this problem so that people can better memorize a scripture, write a program that displays the full scripture and then hides a few words at a time until the complete scripture is hidden.

Specification

Functional requirements

Your program must do the following:

1. Store a scripture, including both the reference (for example "John 3:16") and the text of the scripture.
2. Accommodate scriptures with multiple verses, such as "Proverbs 3:5-6".
3. Clear the console screen and display the complete scripture, including the reference and the text.
4. Prompt the user to press the enter key or type quit.
5. If the user types quit, the program should end.
6. If the user presses the enter key (without typing quit), the program should hide a few random words in the scripture, clear the console screen, and display the scripture again. (Hiding a word means that the word should be replaced by underscores (`_`) and the number of underscores should match the number of letters in that word.)
7. The program should continue prompting the user and hiding more words until all words in the scripture are hidden.
8. When all words in the scripture are hidden, the program should end. (The final display of the scripture should show the scripture with all words hidden.)

9. When selecting the random words to hide, for the core requirements, you can select any word at random, even if the word was already hidden. (As a stretch challenge, try to randomly select from only those words that are not already hidden.)

Design Requirements

In addition your program must:

1. Use the principles of Encapsulation, including proper use of classes, methods, public/private access modifiers, and follow good style throughout.
2. Contain at least 3 classes in addition to the **Program** class: one for the scripture itself, one for the reference (for example "John 3:16"), and to represent a word in the scripture.
3. Provide multiple constructors for the scripture reference to handle the case of a single verse and a verse range ("Proverbs 3:5" or "Proverbs 3:5-6").

Showing Creativity and Exceeding Requirements

Meeting the core requirements makes your program eligible to receive a 93%. To receive 100% on the assignment, you need to show creativity and exceed these requirements.

Here are some ideas you might consider:

- Think of other challenges that people find when trying to memorize a scripture. Find a way to have your program help with these challenges.
- Have your program work with a library of scriptures rather than a single one. Choose scriptures at random to present to the user.
- Have the program to load scriptures from a files.
- Anything else you can think of!

Report on what you have done to exceed requirements by adding a description of it in a comment in the Program.cs file.

Video Demo

The following video demonstrates the way this program should work:

Direct link: [Scripture Memorizer Demo](https://www.youtube.com/watch?v=...) (2 minutes)



Code Helps

You might find the following code helps useful in this project:

► Clearing the Console

Design

You will work with your team to create a design for this program. Then, you will each write the code for the program individually.

► Final Design (Do not open until after your design meeting)

Develop the Program

In the course repository, find the **ScriptureMemorizer** project in the **week03** folder and write your program there.

Submission

1. Develop the program using the principle of Encapsulation as described above.

2. Make sure to describe anything you have done to exceed the requirements in comments in the Program.cs file.
3. Commit your source code and push it to GitHub.
4. Verify that you can see your updated code at GitHub.
5. In Canvas, submit a link to your GitHub repository. In the submission comment, describe anything you have done to show creativity and exceed the core requirements.

Copyright © Brigham Young University-Idaho | All rights reserved